Git – Commandes essentielles

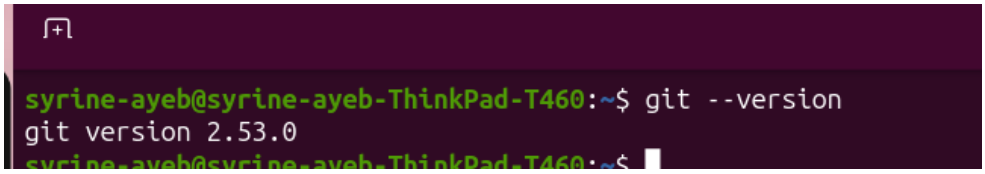
1. Vérifier l'installation de Git

Avant de commencer, vérifier que Git est installé :

git --version

Cette commande affiche la version de Git installée sur votre ordinateur.

Exemple :



2. Configurer Git

Cette configuration est généralement faite une seule fois sur l'ordinateur.

Définir votre nom

git config --global user.name "Votre Nom"

Cette information sera associée à vos commits.

Définir votre email

git config --global user.email "votre@email.com"

Il est recommandé d'utiliser le même email que celui associé à votre compte GitHub.

Vérifier la configuration

git config --global --list

Cette commande affiche les informations de configuration de Git.

3. Créer un dépôt Git

Placez-vous d'abord dans votre projet.

Exemple :

cd university-app

Puis :

git init

Cette commande transforme le dossier du projet en dépôt Git.

Git crée alors un dossier caché .git qui contient les informations nécessaires au suivi des versions.

Vous n'avez normalement pas besoin de modifier ce dossier.

4. Vérifier l'état du projet

git status

Cette commande permet de savoir :

- quels fichiers ont été modifiés ;
- quels fichiers sont nouveaux ;

- quels fichiers sont prêts à être commités ;
- sur quelle branche vous travaillez.

C'est une commande très importante.

Prenez l'habitude de faire :

git status

avant et après vos opérations Git.

5. Ajouter des fichiers

Ajouter tous les fichiers

git add .

Le . signifie : ajouter tous les fichiers modifiés ou nouveaux du projet.

Ajouter un seul fichier

git add src/App.jsx

Cette commande ajoute uniquement le fichier indiqué.

Ajouter plusieurs fichiers

git add src/App.jsx src/App.css

6. Créer un commit

Après avoir ajouté les fichiers :

git commit -m "Create student profile"

Un commit représente une version sauvegardée de votre projet.

Le message doit expliquer clairement ce que vous avez fait.

Évitez les messages trop vagues comme :

git commit -m "update"

Préférez :

git commit -m "Update student profile"

7. Voir l'historique des commits

git log

Cette commande affiche l'historique détaillé des commits.

Pour une version plus simple :

git log --oneline

Exemple :

a52c123 Update student profile
91bd456 Create student profile

8. Créer le dépôt sur GitHub

Après avoir créé votre compte GitHub, créez un nouveau repository.

Exemple de nom :

university-app

Pour un premier TP, il est préférable de créer un repository vide sur GitHub.

9. Connecter le projet local à GitHub

Dans le terminal, utilisez :

git remote add origin https://github.com/USERNAME/university-app.git

Remplacez USERNAME par votre nom d'utilisateur GitHub.

origin est simplement le nom donné au dépôt distant.

10. Vérifier la connexion avec GitHub

git remote -v

Cette commande affiche l'adresse du repository GitHub connecté au projet.

Vous devriez voir quelque chose comme :

origin https://github.com/USERNAME/university-app.git (fetch)
origin https://github.com/USERNAME/university-app.git (push)

11. Renommer la branche principale

Pour utiliser main comme branche principale :

git branch -M main

Cette commande renomme la branche actuelle en main.

12. Envoyer le projet vers GitHub

Pour le premier envoi :

git push -u origin main

Cette commande envoie votre branche main vers GitHub.

L'option -u permet d'associer la branche locale à la branche distante.

Après cette première commande, vous pourrez généralement utiliser simplement :

git push

13. Le workflow quotidien

Après avoir modifié votre projet :

Étape 1 – Vérifier les modifications

git status

Étape 2 – Ajouter les modifications

git add .

Étape 3 – Créer un commit

git commit -m "Update student profile"

Étape 4 – Envoyer vers GitHub

git push

À retenir

git status
↓
git add .
↓
git commit -m "message"
↓
git push

14. Récupérer les modifications depuis GitHub

git pull

Cette commande récupère les dernières modifications du repository GitHub et les applique à votre projet local.

Utilisez-la notamment lorsque plusieurs personnes travaillent sur le même projet ou lorsque le repository GitHub a été modifié depuis votre dernier téléchargement.

15. Télécharger un projet GitHub

Pour récupérer un projet qui existe déjà sur GitHub :

git clone https://github.com/USERNAME/university-app.git

Git crée automatiquement un dossier contenant le projet.

Ensuite :

cd university-app

Puis, pour un projet React/Vite :

npm install

Enfin :

npm run dev

16. Vérifier les branches

git branch

Cette commande affiche les branches locales.

Exemple :

* main

Le * indique la branche actuellement utilisée.

17. Créer une nouvelle branche

git switch -c feature/student-card

Cette commande :

1. crée une nouvelle branche ;
2. vous déplace automatiquement sur cette branche.

Pour un TP débutant, cette commande est facultative.

18. Changer de branche

git switch main

Cette commande permet de revenir sur la branche main.

19. Supprimer une branche locale

Après avoir terminé une branche :

git branch -d feature/student-card

Cette commande supprime la branche locale.

20. Annuler les modifications d'un fichier non commit

Si vous avez modifié un fichier mais que vous voulez revenir à la dernière version sauvegardée par Git :

git restore filename

Exemple :

git restore src/App.jsx

Attention : les modifications non sauvegardées dans un commit peuvent être perdues.

21. Retirer un fichier de la zone de préparation

Si vous avez utilisé :

git add .

mais que vous ne voulez finalement pas ajouter un fichier au prochain commit :

git restore --staged filename

Exemple :

git restore --staged src/App.css

22. Voir les différences

Pour voir les modifications qui n'ont pas encore été ajoutées :

git diff

Cette commande permet de comparer l'état actuel des fichiers avec leur dernière version suivie par Git.

23. Le fichier .gitignore

Dans un projet React/Vite, certains fichiers ne doivent pas être envoyés vers GitHub.

Le fichier :

.gitignore

indique à Git quels fichiers ou dossiers doivent être ignorés.

Dans un projet React, on retrouve notamment :

node_modules/
dist/
.env

Pourquoi node_modules ?

node_modules contient les dépendances installées par npm.

Il est inutile de l'envoyer vers GitHub car il peut être recréé avec :

npm install

24. Vérifier les fichiers ignorés

Vous pouvez utiliser :

git status

Si .gitignore fonctionne correctement, node_modules ne doit normalement pas apparaître comme une liste énorme de fichiers à ajouter.

25. Supprimer un fichier du projet avec Git

Pour supprimer un fichier :

git rm filename

Puis :

git commit -m "Remove unused file"

26. Renommer un fichier avec Git

git mv old-name.jsx new-name.jsx

Puis :

git commit -m "Rename component"

27. Voir les informations du repository distant

git remote -v

Pour obtenir plus d'informations :

git remote show origin

28. Mettre à jour un projet avant de travailler

Lorsque vous travaillez sur un projet qui existe déjà sur GitHub :

git pull

Puis vous pouvez commencer à travailler.

À la fin :

git add .
git commit -m "Describe your changes"
git push

32. Les 4 commandes essentielles

Pour les TPs, mémorisez surtout :

git status

git add .

git commit -m "message"

git push

Signification

STATUS => Qu'est-ce qui a changé ?
ADD => Je prépare mes changements.

COMMIT => Je sauvegarde cette version.
PUSH => J'envoie cette version sur GitHub.

34. En cas de problème

Ne lancez pas des commandes Git au hasard.

Commencez toujours par :

git status

Puis regardez le message affiché par Git.

Vous pouvez également vérifier le repository distant :

git remote -v

Et l'historique :

git log --oneline

Si vous ne comprenez pas l'erreur, gardez le message affiché dans le terminal afin de pouvoir l'analyser.